

Full Stack Development Week 07

Topics covered:

- useReducer
- useMemo
- Accessing your Cloud SQL database

Activity 1: useReducer

As discussed in last week's lecture, the useReducerhook is a powerful hook in React that's particularly useful when you have complex state logic.

Why use useReducer?

The useReducer hook is beneficial when dealing with complex state logic involving multiple fields, provides predictable state updates through a reducer function, and helps centralize state management logic.

Practical Example: Shopping Cart

Let's create a simple shopping cart using useReducer.

We should have 3 actions:

- ADD_ITEM
- REMOVE_ITEM
- CLEAR_CART

Exercise

Review the activity code and modify it to:

- Add a quantity field to each item
- Implement an "UPDATE_QUANTITY" action
- Display the quantity against the items To discuss
 - Why is useReducer better than useState for this shopping cart example?
 - How would you handle async operations (like fetching product data) with useReducer?

Activity 2: useMemo

The useMemo hook is used for performance optimization by memoizing expensive computations. It only recomputes the memoized value when one of the dependencies has changed.

Why use useMemo?

- Prevents unnecessary recalculations of expensive computations
- Optimizes performance for "expensive" (in terms of time) functions

Exercise

Let's review a simple activity where we have a very large list of items (10,000+) and we need to search for them, plus we also have a counter on the screen as well. Let's see how useMemo makes a difference by creating two examples

To discuss:

- What happens if we increase the counters value, will all 10,000 items re-render if we are not using useMemo
- What's the performance impact of removing useMemo in this example?
- Are there cases where using useMemo might actually hurt performance?

Activity 3: Access your Cloud SQL database

Assignment 2 will require you to save web application data in database tables in addition to localStorage.

For this course, we will use the MS SQL Server database hosted in the Cloud - this is so that you DO NOT HAVE to install it on your machine. While we will start looking at code interacting with the database using ORM next week, it is important that all of you get accustomed to your MS SQL Server database account and logistics around it.

Every student is given their own database instance in the cloud. You may use any VS Code extension to connect to your database. The following instruction is using [SQL Server \(mssql\) - Visual Studio Marketplace](#).

Step 1: SQL Server extension

In your VS Code, download, install and activate SQL Server (mssql) extension.

Step 2: Connect to database

Connect to database using the following credentials

Login Details

Server:	<i>dipto-database.cn2ems8y2mfe.ap-southeast-2.rds.amazonaws.com</i>
User name:	Your student number with pre-pended s
Password:	<i>COSC2758-2026@!</i>
Database name:	The same as your user name

 Connection Dialog ✕



Connect to Database

Profile Name

Connection Group

<Default> ✕

Input type

☒  Parameters [Load from Connection String](#)

☐  Browse Azure

☐  Browse Fabric

Server name * 

dipto-database.cn2ems8y2mfe.ap-southeast-2.rds.amazonaws.com

Trust server certificate 

☐

Authentication type * 

SQL Login ▼

User name * 

s1234567

Password * 

COSC2758-2026@! 

Save Password

☐

Database name 

s1234567

Encrypt 

Mandatory ▼

Advanced

Connect

Step 3: Change your password

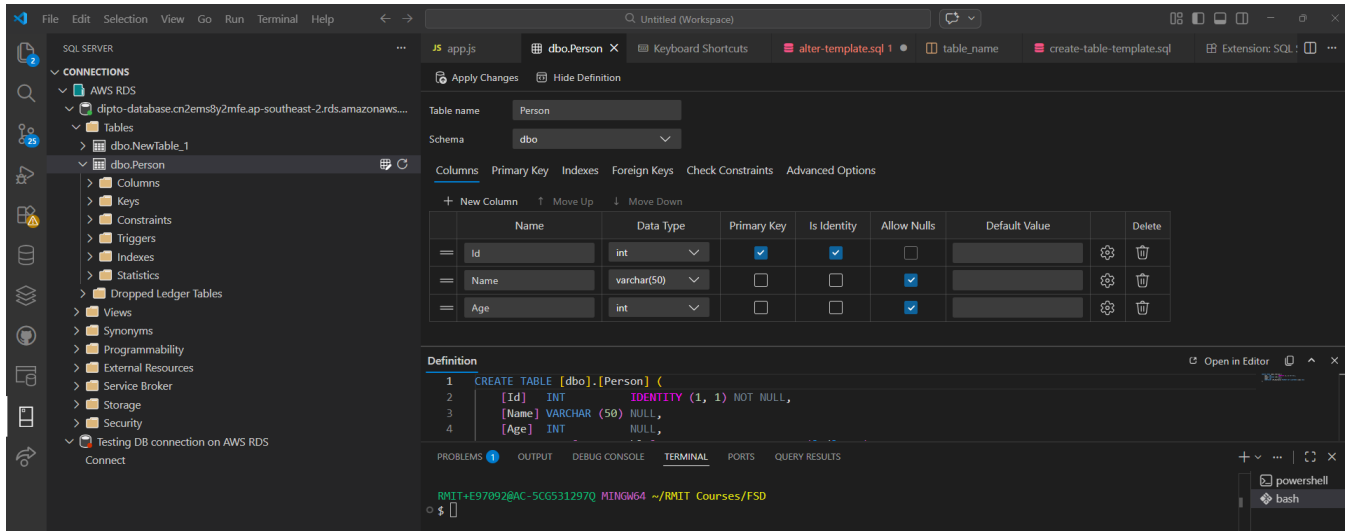
You only need to change it once. Please note down your new password.

A screenshot of the 'Connect to Database' dialog box in VS Code. The dialog is titled 'Change Password' and shows the current password being changed. The 'Username' field is filled with 'diptoStudent'. The 'New Password' and 'Confirm Password' fields are empty. The 'Change Password' button is highlighted.

Note: If you are unable to login using the above, please contact dipto.pratyaksa@rmit.edu.au
Note: Please do not contact ITS for access, this database is maintained by us for this course only.

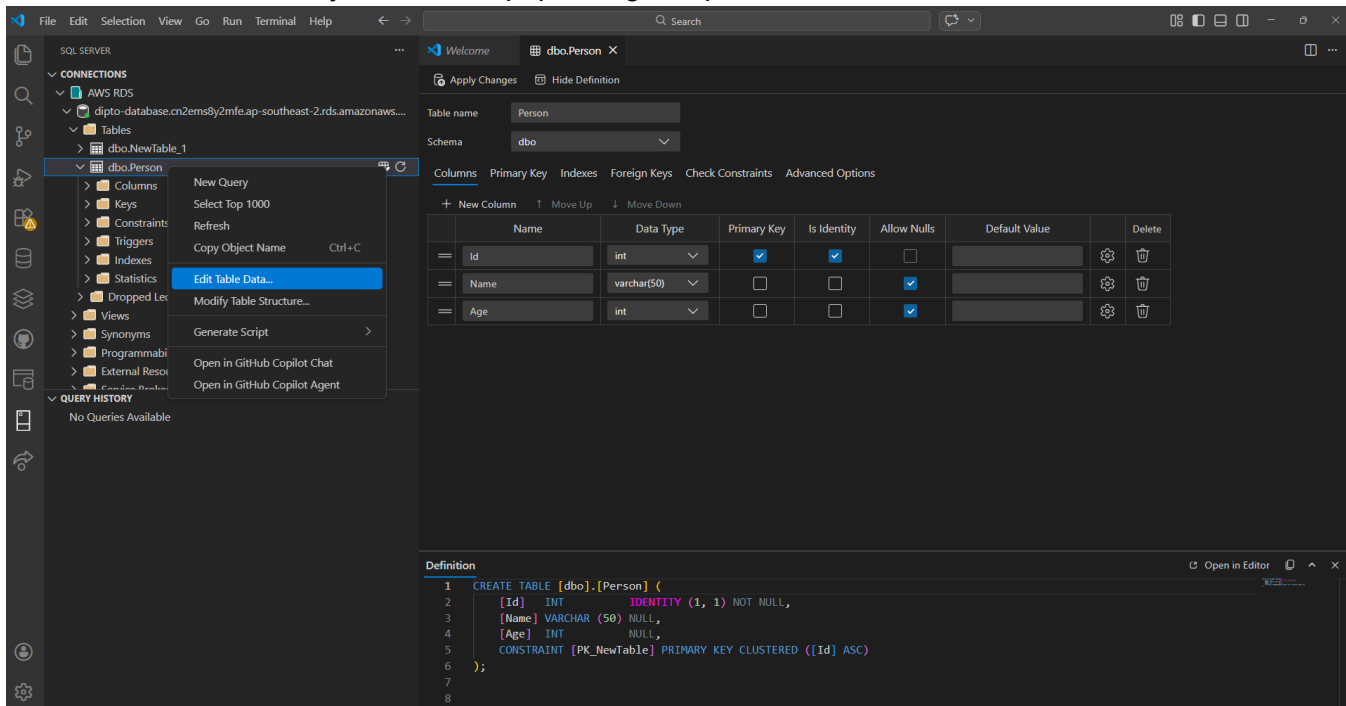
Step 4: Visually create a table

- Right click on the Tables folder and select New Table.
- Give this table a name and add some new columns.
- Apply Changes



Step 5: Edit Table Data

Once the table is saved, you can start populating sample data.



Activity 4: Using SQL

Once data is populated in the Person's table you can start making queries on the SQL tab which can be opened by right-clicking on `dbo.Person` and New Query.

You can experiment with queries such as:

```
SELECT * FROM dbo.Person;
```

```
SELECT Name FROM dbo.Person WHERE Age >= 18;
```

Execute query by clicking the green arrow button or Ctrl+Shift+E

You should see the relevant results.

Once you get familiar with the interface please download and open `activity4.sql` (supplied in this week's prac zip file) on VS Code SQL Server SQL tab. You should a list of SQL statements ready to run. Select all the script (Ctrl+Shift+A) and execute all the queries (Ctrl+Shift+E).

Once this is done, refresh the Tables folder page and you should now see three tables:

- customers
- orders
- order_items

Explore these three tables, seeing if you can write a query that pulls out a list of items ordered by *Alice* across all their orders.

```
SELECT * FROM orders JOIN customers  
ON orders.customer_id = customers.customer_id  
WHERE first_name = 'Alice'
```